



Foundation University Islamabad

School of Science and Technology

Name: Hunaina Yasir

Subject: OOP LAB

Roll Number: 073

Section: B

LAB REPORT NO.5

TASK 1:

In a university system, a Professor plays a dual role as both a Teacher and a Researcher. A Teacher specializes in a particular subject, while a Researcher focuses on a specific research field. Using the concept of multiple inheritance in C++, design a program where the Teacher class has a parameterized constructor to initialize the subject taught, and the Researcher class has a parameterized constructor to initialize the research field. The Professor class should inherit from both Teacher and Researcher and have an additional parameter to store the years of experience. Implement constructor initialization lists in the Professor class to initialize all inherited attributes. In the main() function, create an instance of the Professor class and display the details of the professor, including the subject they teach, their research field, and their years of experience.

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Teacher {
protected:
    string subject;

public:
    Teacher(string sub) {
        subject = sub;
    }
};
```

```
class Researcher {
protected:
    string researchField;

public:
    Researcher(string field) {
        researchField = field;
    }
};
```

```

    }
};

class Professor : public Teacher, public Researcher {
private:
    int experience;

public:
    Professor(string sub, string field, int exp) : Teacher(sub), Researcher(field) {
        experience = exp;
    }

    void displayProfessor() {
        cout << "Subject: " << subject << endl;
        cout << "Research Field: " << researchField << endl;
        cout << "Years of Experience: " << experience << endl;
    }
};

int main() {
    Professor p1("Computer Science", "Artificial Intelligence", 12);
    p1.displayProfessor();

    return 0;
}

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class Teacher {
6  protected:
7      string subject;
8
9  public:
10     Teacher(string sub) {
11         subject = sub;
12     }
13 };
14
15 class Researcher {
16 protected:
17     string researchField;
18
19 public:
20     Researcher(string field) {
21         researchField = field;
22     }
23 };
24
25 class Professor : public Teacher, public Researcher {
26 private:
27     int experience;
28
29 public:
30     Professor(string sub, string field,
31 int exp) : Teacher(sub), Researcher(field) {
32         experience = exp;
33     }
34

```

```

24
25 class Professor : public Teacher, public Researcher {
26 private:
27     int experience;
28
29 public:
30     Professor(string sub, string field,
31 int exp) : Teacher(sub), Researcher(field) {
32         experience = exp;
33     }
34
35     void displayProfessor() {
36         cout << "Subject: " << subject << endl;
37         cout << "Research Field: " << researchField << endl;
38         cout << "Years of Experience: " << experience << endl;
39     }
40 };
41
42 int main() {
43     Professor p1("Computer Science", "Artificial Intelligence", 12);
44     p1.displayProfessor();
45
46     return 0;
47 }

```

OUTPUT:

Subject: Computer Science
Research Field: Artificial Intelligence
Years of Experience: 12

Subject: Computer Science
Research Field: Artificial Intelligence
Years of Experience: 12

TASK 2:

A Vehicle can have different types, such as cars and trucks, and it can belong to a particular brand. In a showroom, each Car is identified by its type and brand and has a specific model name. Using multiple inheritance in C++, create a program where the Vehicle class has a parameterized constructor to initialize the vehicle type, and the Brand class has a parameterized constructor to store the brand name. The Car class should inherit from both Vehicle and Brand, and it should have an additional parameter for the model name. Use constructor initialization lists in the Car class to initialize all attributes. In the main() function, create an instance of the Car class and display the car's details, including its type, brand, and model name.

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;

class Vehicle {
protected:
    string type;

public:
    Vehicle(string t) {
        type = t;
    }
};

class Brand {
protected:
    string brandName;

public:
    Brand(string b) {
        brandName = b;
    }
};

class Car : public Vehicle, public Brand {
private:
    string modelName;

public:
    Car(string t, string b, string m) : Vehicle(t), Brand(b) {
```

```

        modelName = m;
    }

    void displayCar() {
        cout << "Vehicle Type: " << type << endl;
        cout << "Brand: " << brandName << endl;
        cout << "Model: " << modelName << endl;
    }
};

int main() {
    Car c1("SUV", "Ford", "Explorer");
    c1.displayCar();

    return 0;
}

```

```

5  class Vehicle {
6      protected:
7          string type;
8
9      public:
10         Vehicle(string t) {
11             type = t;
12         }
13     };
14
15     class Brand {
16     protected:
17         string brandName;
18
19     public:
20         Brand(string b) {
21             brandName = b;
22         }
23     };
24
25     class Car : public Vehicle, public Brand {
26     private:
27         string modelName;
28
29     public:
30         Car(string t, string b, string m) : Vehicle(t), Brand(b) {
31             modelName = m;
32         }
33         void displayCar() {
34             cout << "Vehicle Type: " << type << endl;
35             cout << "Brand: " << brandName << endl;
36             cout << "Model: " << modelName << endl;
37         }
38     };
39
40     int main() {
41         Car c1("SUV", "Ford", "Explorer");
42         c1.displayCar();
43         return 0;
44     }

```

OUTPUT:

Vehicle Type: SUV
Brand: Ford
Model: Explorer

```

Vehicle Type: SUV
Brand: Ford
Model: Explorer

```

TASK 3:

In a hospital management system, doctors serve as both medical professionals and employees of the hospital. A MedicalProfessional has a specialization (e.g., "Cardiologist"), while an Employee has an employee ID assigned by the hospital. A Doctor class should inherit from both MedicalProfessional and Employee and should also have an additional parameter for the doctor's years of experience. Using multiple inheritance in C++, design a program where the MedicalProfessional class has a parameterized constructor for specialization, and the Employee class has a parameterized constructor for employee ID. The Doctor class should use constructor initialization lists to initialize all inherited attributes. In the main() function, create an instance of the Doctor class and display the doctor's specialization, employee ID, and years of experience.

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;

class MedicalProfessional {
protected:
    string specialization;

public:
    MedicalProfessional(string spec) {
        specialization = spec;
    }
};

class Employee {
protected:
    int employeeID;

public:
    Employee(int id) {
        employeeID = id;
    }
};

class Doctor : public MedicalProfessional, public Employee {
private:
    int experience;

public:
    Doctor(string spec, int id, int exp) : MedicalProfessional(spec), Employee(id) {
        experience = exp;
    }

    void showDoctor() {
        cout << "Specialization: " << specialization << endl;
        cout << "Employee ID: " << employeeID << endl;
    }
};
```

```

        cout << "Experience: " << experience << " years" << endl;
    }
};

int main() {
    Doctor d1("Cardiologist", 9002, 15);
    d1.showDoctor();

    return 0;
}

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class MedicalProfessional {
6  protected:
7      string specialization;
8
9  public:
10     MedicalProfessional(string spec) {
11         specialization = spec;
12     }
13 };
14
15 class Employee {
16 protected:
17     int employeeID;
18
19 public:
20     Employee(int id) {
21         employeeID = id;
22     }
23 };
24
25 class Doctor : public MedicalProfessional, public Employee {
26 private:
27     int experience;
28
29 public:
30     Doctor(string spec, int id, int exp) : MedicalProfessional(spec)
31     , Employee(id) {
32         experience = exp;
33     }
34
35     void showDoctor() {
36         cout << "Specialization: " << specialization << endl;
37         cout << "Employee ID: " << employeeID << endl;
38         cout << "Experience: " << experience << " years" << endl;
39     }

```

```

29 public:
30     Doctor(string spec, int id, int exp) : MedicalProfessional(spec)
31     , Employee(id) {
32         experience = exp;
33     }
34
35     void showDoctor() {
36         cout << "Specialization: " << specialization << endl;
37         cout << "Employee ID: " << employeeID << endl;
38         cout << "Experience: " << experience << " years" << endl;
39     }
40 };
41
42 int main() {
43     Doctor d1("Cardiologist", 9002, 15);
44     d1.showDoctor();
45
46     return 0;
47 }

```

OUTPUT:

Specialization: Cardiologist
Employee ID: 9002
Experience: 15 years

```

Specialization: Cardiologist
Employee ID: 9002
Experience: 15 years

```